

Guía Didáctica: Patrón DTO - Data Transfer Object

Índice

1. [¿Qué es el Patrón DTO?](#)
2. [¿Por qué es necesario? El Problema de la Exposición Directa](#)
3. [DTO vs Entidad: La Diferencia Clave](#)
4. [Tipos de DTOs](#)
5. [Implementación en TypeScript](#)
6. [Caso de Estudio: Concesionaria de Vehículos](#)
7. [DTO en Arquitectura Hexagonal](#)
8. [Validación de DTOs](#)
9. [Transformación: Mapeo de DTOs](#)
10. [Ventajas y Desventajas](#)
11. [Anti-patrones: Qué NO Hacer](#)
12. [Ejercicios Prácticos](#)
13. [Checklist de Implementación](#)

1. ¿Qué es el Patrón DTO?

DEFINICIÓN

DTO (DATA TRANSFER OBJECT)

=====

Es un objeto que transporta datos entre diferentes capas de una aplicación sin comportamiento (sin métodos de negocio). Su único propósito es contener datos para ser transferidos.



ANALOGÍA: SOBRE DE CORREO



Sobre
(DTO)



Carta
(Datos)



Remitente
(Capa origen)



Destinatario
(Capa destino)

El Patrón en Una Frase

"Un DTO es un contenedor de datos sin inteligencia, diseñado para mover información entre capas de forma eficiente y segura."

DTO EN POCAS PALABRAS

❌ SIN DTO (Exponiendo entidad directamente):

```
app.get('/api/empleados/:id', (req, res) => {
  const emp = repository.findById(req.params.id);
  res.json(emp); // ← Expone TODO: id, createdAt, etc.
});
```

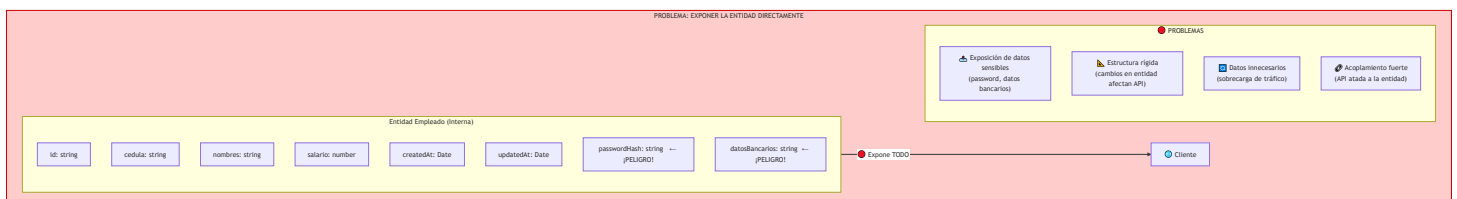
✅ CON DTO:

```
app.get('/api/empleados/:id', (req, res) => {
  const emp = repository.findById(req.params.id);
  const dto = EmployeeResponse.fromEntity(emp);
  res.json(dto); // ← Solo lo que queremos exponer
});
```

🎯 SEPARAMOS LO QUE ES INTERNO DE LO QUE SE EXPONE

2. ¿Por qué es necesario? El Problema de la Exposición Directa

El Problema Visual



Ejemplo Concreto del Problema

```
// ❌ CÓDIGO SIN DTO (EL PROBLEMA)

// Entidad de dominio (rica en comportamiento y datos)
class Employee {
    private _id: string;
    private _cedula: string;
    private _nombres: string;
    private _salario: number;
    private _createdAt: Date;
    private _updatedAt: Date;
    private _passwordHash: string; // ← ¡SENSIBLE!
    private _cuentaBancaria: string; // ← ¡SENSIBLE!
    private _departamento: string;

    // ... Métodos de negocio ...

    // ❌ Método toJSON() EXPONE TODO
    toJSON(): any {
        return {
            id: this._id,
            cedula: this._cedula,
            nombres: this._nombres,
            salario: this._salario,
            createdAt: this._createdAt,
            updatedAt: this._updatedAt,
            passwordHash: this._passwordHash, // ← EXPUESTO!
            cuentaBancaria: this._cuentaBancaria, // ← EXPUESTO!
            departamento: this._departamento
        };
    }
}

// ❌ Controller expone TODA la entidad
app.get('/api/employees/:id', (req, res) => {
    const employee = employeeService.findById(req.params.id);
    // ❌ El cliente recibe TODOS los datos, incluyendo passwordHash!
    res.json(employee.toJSON());
});
```

Problemas Identificados

Problema	Manifestación	Impacto
Exposición de datos sensibles	passwordHash, cuentaBancaria expuestos	 Riesgo de seguridad
Acoplamiento API-Entidad	Cambios en entidad afectan API	 Breaking changes
Sobrecarga de datos	Envío de campos no necesarios	 Ineficiencia
Falta de flexibilidad	No se pueden estructurar datos diferente	 Rigidez
Dificultad de versionado	No se puede tener v1 y v2 de API	 Problemas de evolución

3. DTO vs Entidad: La Diferencia Clave



Comparativa Detallada

Aspecto	Entidad de Dominio	DTO
Propósito	Representar el negocio	Transferir datos
Comportamiento	Tiene métodos de negocio	Solo datos

Aspecto	Entidad de Dominio	DTO
Reglas de negocio	✅ Contiene reglas	❌ Sin reglas
Identidad	✅ Tiene ID único	❌ No tiene identidad
Inmutabilidad	⚠️ Puede ser mutable	✅ Siempre inmutable
Validaciones	✅ Validaciones de dominio	⚠️ Solo formato básico
Ubicación	Capa de dominio	Capa de aplicación
Serialización	⚠️ No es segura	✅ Diseñada para serializar

Ejemplo Comparativo

```
// =====
// ENTIDAD DE DOMINIO (con comportamiento)
// =====

class Employee {
    private _id: string;
    private _cedula: string;
    private _nombres: string;
    private _salario: number;
    private _createdAt: Date;
    private _updatedAt: Date;

    // ✅ Constructor con validaciones
    private constructor(data: EmployeeData) {
        this._id = data.id;
        this._cedula = data.cedula;
        this._nombres = data.nombres;
        this._salario = data.salario;
        this._createdAt = data.createdAt;
        this._updatedAt = data.updatedAt;
        this.validate();
    }

    // ✅ Factory methods con reglas de negocio
    static create(cedula: string, nombres: string, salario: number): Employee {
        const id = generateId();
        const now = new Date();
        return new Employee({ id, cedula, nombres, salario, createdAt: now, updatedAt: now });
    }

    // ✅ Métodos de comportamiento
    changeSalario(nuevoSalario: number): void {
        if (nuevoSalario < 0) throw new Error('Salario inválido');
        if (nuevoSalario > 1000000) throw new Error('Salario excede límite');
        this._salario = nuevoSalario;
        this._updatedAt = new Date();
    }

    // ✅ Getters
    getId(): string { return this._id; }
    getCedula(): string { return this._cedula; }
    getNombres(): string { return this._nombres; }
```

```

    getSalario(): number { return this._salario; }
}

// =====
// DTO (solo datos, sin comportamiento)
// =====

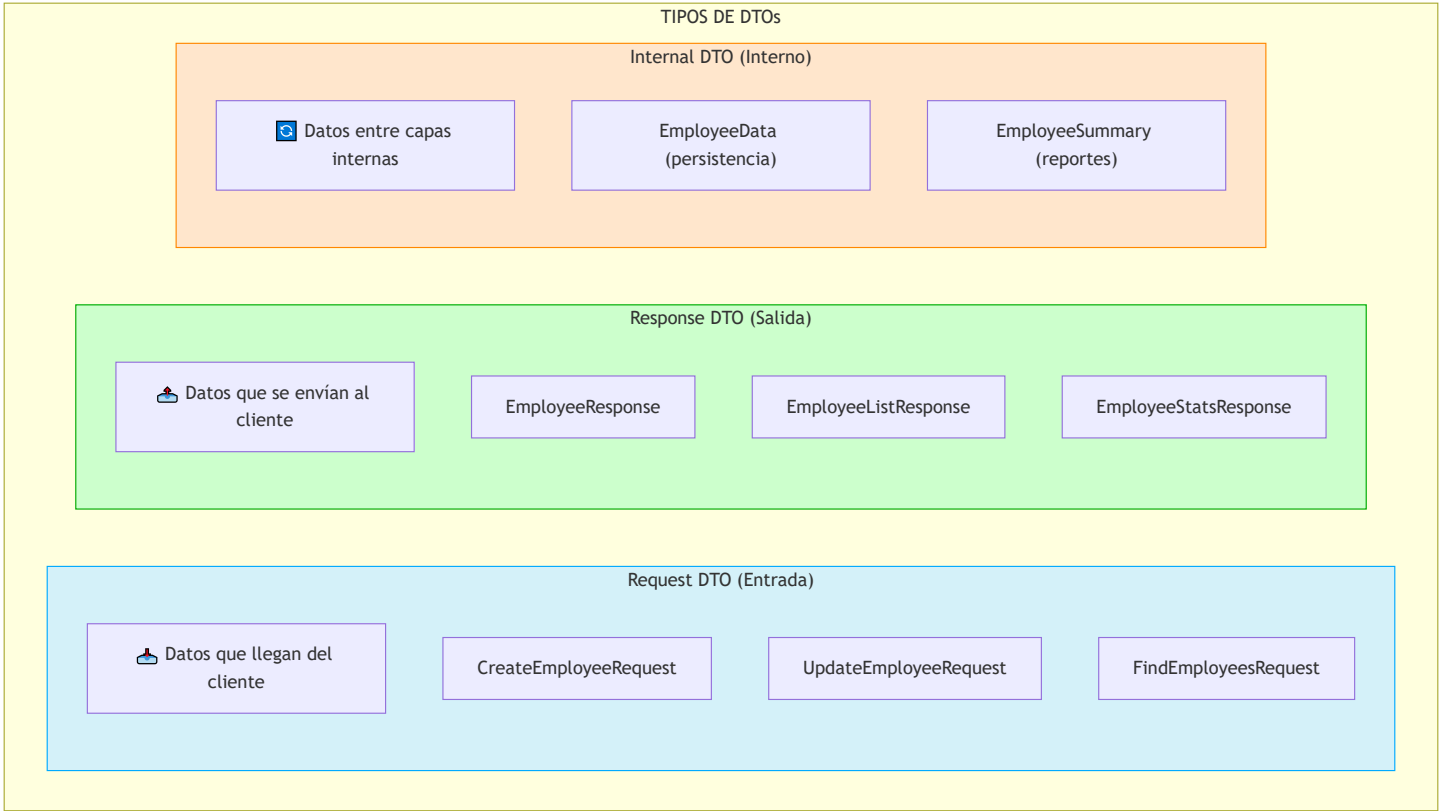
// Request DTO (lo que viene del cliente)
interface CreateEmployeeRequest {
    cedula: string;
    nombres: string;
    salario: number;
    // ❌ No tiene id, createdAt, updatedAt
}

// Response DTO (lo que se envía al cliente)
interface EmployeeResponse {
    id: string;
    cedula: string;
    nombres: string;
    salario: number;
    // ❌ No tiene createdAt, updatedAt
    // ❌ No tiene métodos
}

// ✅ Mapper: Entidad → DTO (en el Service)
class EmployeeMapper {
    static toResponse(employee: Employee): EmployeeResponse {
        return {
            id: employee.getId(),
            cedula: employee.getCedula(),
            nombres: employee.getNombres(),
            salario: employee.getSalario()
        };
    }
}

```

4. Tipos de DTOs



Request DTOs

```
// 📁 application/dtos/requests/create-employee.request.ts
```

```
/**
 * DTO de SOLICITUD: Datos para crear un empleado
 * El cliente envía estos datos, nosotros los validamos
 */
export interface CreateEmployeeRequest {
  cedula: string;
  nombres: string;
  salario: number;
  departamento?: string; // Opcional
}
```

```
// 📁 application/dtos/requests/update-employee.request.ts
```

```
/**
 * DTO de SOLICITUD: Datos para actualizar un empleado
 * Todos los campos son opcionales (PATCH)
 */
export interface UpdateEmployeeRequest {
  nombres?: string;
  salario?: number;
  departamento?: string;
}
```

```
// 📁 application/dtos/requests/find-employees.request.ts
```

```
/**
 * DTO de SOLICITUD: Filtros para buscar empleados
 * Permite paginación y filtros
 */
export interface FindEmployeesRequest {
  page?: number; // Página (default: 1)
  limit?: number; // Límite por página (default: 10)
  sortBy?: string; // Campo para ordenar
  sortOrder?: 'asc' | 'desc';
  search?: string; // Búsqueda por nombre
  minSalary?: number;
  maxSalary?: number;
  department?: string;
}
```

```
includeInactive?: boolean;
```

```
}
```

Response DTOs

```
// 📁 application/dtos/responses/employee.response.ts
```

```
/**
 * DTO de RESPUESTA: Datos de un empleado (versión pública)
 * No incluye datos sensibles
 */
export interface EmployeeResponse {
  id: string;
  cedula: string;
  nombres: string;
  salario: number;
  departamento: string;
  createdAt: string; // ISO String (no Date)
  updatedAt: string; // ISO String (no Date)
}
```

```
// 📁 application/dtos/responses/employee-list.response.ts
```

```
/**
 * DTO de RESPUESTA: Lista de empleados con metadatos
 */
export interface EmployeeListResponse {
  data: EmployeeResponse[];
  meta: {
    total: number;
    page: number;
    limit: number;
    totalPages: number;
  };
}
```

```
// 📁 application/dtos/responses/employee-stats.response.ts
```

```
/**
 * DTO de RESPUESTA: Estadísticas de empleados
 * Agregación de datos para dashboards
 */
export interface EmployeeStatsResponse {
  total: number;
  averageSalary: number;
  maxSalary: number;
}
```

```
minSalary: number;
byDepartment: Array<{
  department: string;
  count: number;
  averageSalary: number;
}>;
bySalaryRange: Array<{
  range: string; // "0-1000", "1001-3000", etc.
  count: number;
}>;
}
```

Internal DTOs (Entre Capas)

```
// 📁 application/dtos/internal/employee.data.ts
```

```
/**  
 * DTO INTERNO: Datos para persistencia  
 * Usado entre Service y Repository  
 * NO expuesto al cliente  
 */
```

```
export interface EmployeeData {  
  id: string;  
  cedula: string;  
  nombres: string;  
  salario: number;  
  departamento: string;  
  createdAt: Date;  
  updatedAt: Date;  
  // Datos que NO van al cliente  
  passwordHash: string;  
  cuentaBancaria: string;  
  historialCambios: ChangeLog[];  
}
```

```
// 📁 application/dtos/internal/employee-summary.ts
```

```
/**  
 * DTO INTERNO: Resumen para reportes  
 * No incluye todos los datos  
 */  
export interface EmployeeSummary {  
  id: string;  
  nombres: string;  
  salario: number;  
  departamento: string;  
  antigüedad: number; // años en la empresa  
  cumplimientoMeta: boolean;  
}
```


5. Implementación en TypeScript

Estructura Completa de DTOs

```
src/  
└─ application/  
    └─ dtos/  
        ├── index.ts                # Barrel file  
        ├── requests/  
        │   ├── index.ts  
        │   ├── create-employee.request.ts  
        │   ├── update-employee.request.ts  
        │   └─ find-employees.request.ts  
        ├── responses/  
        │   ├── index.ts  
        │   ├── employee.response.ts  
        │   ├── employee-list.response.ts  
        │   └─ employee-stats.response.ts  
        └─ mappers/  
            ├── index.ts  
            └─ employee.mapper.ts
```

Request DTOs Completos

```
// 📁 application/dtos/requests/create-employee.request.ts
```

```
/**
 * CreateEmployeeRequest
 * =====
 * DTO para la creación de un nuevo empleado.
 *
 * @example
 * ```typescript
 * const request: CreateEmployeeRequest = {
 *   cedula: '1234567890',
 *   nombres: 'Ana García',
 *   salario: 1500.00
 * };
 * ```
 */
export interface CreateEmployeeRequest {
  /**
   * Cédula del empleado (mínimo 10 caracteres)
   * @example '1234567890'
   */
  cedula: string;

  /**
   * Nombres completos del empleado
   * @example 'Ana María García Pérez'
   */
  nombres: string;

  /**
   * Salario mensual en dólares (mínimo 0)
   * @example 1500.00
   */
  salario: number;

  /**
   * Departamento opcional
   * @example 'Ventas'
   */
  departamento?: string;
}
```

```
// 📁 application/dtos/requests/update-employee.request.ts
```

```
/**
 * UpdateEmployeeRequest
 * =====
 * DTO para actualizar un empleado existente.
 * Todos los campos son opcionales (PATCH).
 */
```

```
export interface UpdateEmployeeRequest {
  /** Nuevos nombres del empleado */
  nombres?: string;

  /** Nuevo salario del empleado */
  salario?: number;

  /** Nuevo departamento */
  departamento?: string;

  /** Estado activo/inactivo */
  activo?: boolean;
}
```

```
// 📁 application/dtos/requests/find-employees.request.ts
```

```
/**
 * FindEmployeesRequest
 * =====
 * DTO para filtrar y paginar la búsqueda de empleados.
 */
```

```
export interface FindEmployeesRequest {
  /** Página actual (default: 1) */
  page?: number;

  /** Elementos por página (default: 10, max: 100) */
  limit?: number;

  /** Campo por el cual ordenar */
  sortBy?: 'id' | 'nombres' | 'salario' | 'createdAt';

  /** Dirección del orden */
  sortOrder?: 'asc' | 'desc';
}
```

```
/** Texto de búsqueda en nombres y cédula */
```

```
search?: string;
```

```
/** Filtro por salario mínimo */
```

```
minSalary?: number;
```

```
/** Filtro por salario máximo */
```

```
maxSalary?: number;
```

```
/** Filtro por departamento */
```

```
departamento?: string;
```

```
/** Incluir empleados inactivos */
```

```
includeInactive?: boolean;
```

```
}
```

Response DTOs Completos

```
// 📁 application/dtos/responses/employee.response.ts
```

```
/**
 * EmployeeResponse
 * =====
 * DTO para la respuesta de un empleado.
 * Es la versión PÚBLICA de los datos.
 */
export interface EmployeeResponse {
  /** Identificador único del empleado */
  id: string;

  /** Cédula del empleado (últimos 4 dígitos ocultos en producción) */
  cedula: string;

  /** Nombres completos */
  nombres: string;

  /** Salario mensual */
  salario: number;

  /** Departamento al que pertenece */
  departamento: string;

  /** Fecha de creación (ISO 8601) */
  createdAt: string;

  /** Fecha de última actualización (ISO 8601) */
  updatedAt: string;

  /** Indicador de empleado activo */
  activo: boolean;
}
```

```
// 📁 application/dtos/responses/employee-list.response.ts
```

```
/**
 * EmployeeListResponse
 * =====
 * DTO para listar empleados con metadatos de paginación.
 */
```

```

export interface EmployeeListResponse {
  /** Lista de empleados */
  data: EmployeeResponse[];

  /** Metadatos de paginación */
  meta: {
    /** Total de registros disponibles */
    total: number;

    /** Página actual */
    page: number;

    /** Límite por página */
    limit: number;

    /** Total de páginas */
    totalPages: number;
  };
}

// 📁 application/dtos/responses/employee-stats.response.ts

/**
 * EmployeeStatsResponse
 * =====
 * DTO para estadísticas agregadas de empleados.
 */
export interface EmployeeStatsResponse {
  /** Total de empleados */
  total: number;

  /** Salario promedio */
  averageSalary: number;

  /** Salario más alto */
  maxSalary: number;

  /** Salario más bajo */
  minSalary: number;

  /** Distribución por departamento */
  byDepartment: Array<{
    departamento: string;

```

```
        count: number;
        averageSalary: number;
    }>;

    /** Distribución por rango salarial */
    bySalaryRange: Array<{
        range: string; // "0-1000", "1001-3000", etc.
        count: number;
    }>;
}
```

Mappers: Transformación Entidad ⇌ DTO

```
// 📁 application/dtos/mappers/employee.mapper.ts
```

```
import { Employee } from '../../../domain/entities/employee.entity.js';
import { CreateEmployeeRequest, UpdateEmployeeRequest } from '../requests/index.js';
import { EmployeeResponse, EmployeeListResponse, EmployeeStatsResponse } from '../responses/index.js';

/**
 * EmployeeMapper
 * =====
 * Responsable de transformar entre Entidades y DTOs.
 */
export class EmployeeMapper {

    // =====
    // REQUEST → DOMAIN
    // =====

    /**
     * Convierte un CreateEmployeeRequest a datos para la entidad
     */
    static toEntityData(request: CreateEmployeeRequest): {
        cedula: string;
        nombres: string;
        salario: number;
        departamento: string;
    } {
        return {
            cedula: request.cedula.trim(),
            nombres: request.nombres.trim(),
            salario: request.salario,
            departamento: request.departamento || 'Sin asignar'
        };
    }

    // =====
    // DOMAIN → RESPONSE
    // =====

    /**
     * Convierte una Entidad a EmployeeResponse (público)
     * @param employee - Entidad de dominio
     */
}
```



```

* @returns DTO de respuesta pública
*/
static toResponse(employee: Employee): EmployeeResponse {
    return {
        id: employee.getId(),
        cedula: employee.getCedula(),
        nombres: employee.getNombres(),
        salario: employee.getSalario(),
        departamento: employee.getDepartamento(),
        createdAt: employee.getCreatedAt().toISOString(),
        updatedAt: employee.getUpdatedAt().toISOString(),
        activo: employee.isActive()
    };
}

/**
 * Convierte un array de Entidades a EmployeeListResponse
 */
static toListResponse(
    employees: Employee[],
    total: number,
    page: number,
    limit: number
): EmployeeListResponse {
    return {
        data: employees.map(emp => this.toResponse(emp)),
        meta: {
            total,
            page,
            limit,
            totalPages: Math.ceil(total / limit)
        }
    };
}

/**
 * Convierte datos agregados a EmployeeStatsResponse
 */
static toStatsResponse(data: {
    total: number;
    averageSalary: number;
    maxSalary: number;
    minSalary: number;

```

```

        byDepartment: Array<{ departamento: string; count: number; averageSalary: number }>;
        bySalaryRange: Array<{ range: string; count: number }>;
    }): EmployeeStatsResponse {
        return {
            total: data.total,
            averageSalary: data.averageSalary,
            maxSalary: data.maxSalary,
            minSalary: data.minSalary,
            byDepartment: data.byDepartment,
            bySalaryRange: data.bySalaryRange
        };
    }

// =====
// REQUEST → VALIDACIÓN
// =====

/**
 * Valida un CreateEmployeeRequest
 * @throws {Error} Si el request es inválido
 */
static validateCreateRequest(request: CreateEmployeeRequest): void {
    // Validar cédula
    if (!request.cedula || request.cedula.trim().length < 10) {
        throw new Error('La cédula debe tener al menos 10 caracteres');
    }

    // Validar nombres
    if (!request.nombres || request.nombres.trim().length < 2) {
        throw new Error('Los nombres deben tener al menos 2 caracteres');
    }

    // Validar salario
    if (request.salario < 0) {
        throw new Error('El salario no puede ser negativo');
    }

    if (request.salario > 1000000) {
        throw new Error('El salario no puede exceder $1,000,000');
    }
}

/**

```

```

* Valida un UpdateEmployeeRequest
*/
static validateUpdateRequest(request: UpdateEmployeeRequest): void {
    // Validar que haya al menos un campo para actualizar
    if (!request.nombres && !request.salario && !request.departamento && request.activo ===
        throw new Error('Debe proporcionar al menos un campo para actualizar');
    }

    // Validar nombres si está presente
    if (request.nombres && request.nombres.trim().length < 2) {
        throw new Error('Los nombres deben tener al menos 2 caracteres');
    }

    // Validar salario si está presente
    if (request.salario !== undefined) {
        if (request.salario < 0) {
            throw new Error('El salario no puede ser negativo');
        }
        if (request.salario > 1000000) {
            throw new Error('El salario no puede exceder $1,000,000');
        }
    }
}

/**
* Valida un FindEmployeesRequest
*/
static validateFindRequest(request: FindEmployeesRequest): void {
    // Validar paginación
    if (request.page && request.page < 1) {
        throw new Error('La página debe ser mayor o igual a 1');
    }

    if (request.limit && (request.limit < 1 || request.limit > 100)) {
        throw new Error('El límite debe estar entre 1 y 100');
    }

    // Validar filtros de salario
    if (request.minSalary && request.minSalary < 0) {
        throw new Error('El salario mínimo no puede ser negativo');
    }

    if (request.maxSalary && request.maxSalary < 0) {

```

```
        throw new Error('El salario máximo no puede ser negativo');
    }

    if (request.minSalary && request.maxSalary && request.minSalary > request.maxSalary) {
        throw new Error('El salario mínimo no puede ser mayor que el máximo');
    }
}
}
```

6. Caso de Estudio: Concesionaria de Vehículos

Contexto

La concesionaria necesita una API REST para gestionar vehículos, cotizaciones y ventas con diferentes niveles de acceso.

DTOs para el Sistema de Concesionaria

```
// =====  
// REQUESTS DTOs  
// =====  
  
// 📁 application/dtos/requests/create-vehiculo.request.ts  
  
export interface CreateVehiculoRequest {  
  marca: string;  
  modelo: string;  
  anio: number;  
  precioBase: number;  
  tipo: 'SEDAN' | 'SUV' | 'HATCHBACK' | 'PICKUP' | 'DEPORTIVO';  
  stock: number;  
  especificaciones?: {  
    motor?: string;  
    transmision?: string;  
    color?: string;  
  };  
}  
  
// 📁 application/dtos/requests/create-cotizacion.request.ts  
  
export interface CreateCotizacionRequest {  
  vehiculoId: string;  
  planId: string;  
  entradaMonto: number;  
  plazoMeses: number;  
  cliente: {  
    nombre: string;  
    email: string;  
    telefono: string;  
    identificacion: string;  
  };  
}  
  
// =====  
// RESPONSES DTOs  
// =====  
  
// 📁 application/dtos/responses/vehiculo.response.ts
```

```

export interface VehiculoResponse {
  id: string;
  marca: string;
  modelo: string;
  anio: number;
  precioBase: number;
  precioFinal: number;
  tipo: string;
  stock: number;
  estado: 'DISPONIBLE' | 'AGOTADO' | 'EN_RESERVA';
  especificaciones: {
    motor?: string;
    transmision?: string;
    color?: string;
  };
}

```

// 📁 application/dtos/responses/cotizacion.response.ts

```

export interface CotizacionResponse {
  id: string;
  fecha: string;
  vehiculo: VehiculoResumen;
  plan: PlanResumen;
  entradaMonto: number;
  plazoMeses: number;
  cuotaMensual: number;
  totalInteres: number;
  totalFinanciado: number;
  cliente: ClienteResumen;
  estado: 'PENDIENTE' | 'APROBADA' | 'RECHAZADA' | 'EXPIRADA';
  tablaAmortizacion: TablaAmortizacionRow[];
}

```

```

// =====
// MAPPER
// =====

```

// 📁 application/dtos/mappers/vehiculo.mapper.ts

```

import { Vehiculo } from '../../../domain/entities/vehiculo.entity.js';
import { VehiculoResponse } from '../../../responses/vehiculo.response.js';
import { Dinero } from '../../../domain/value-objects/dinero.js';

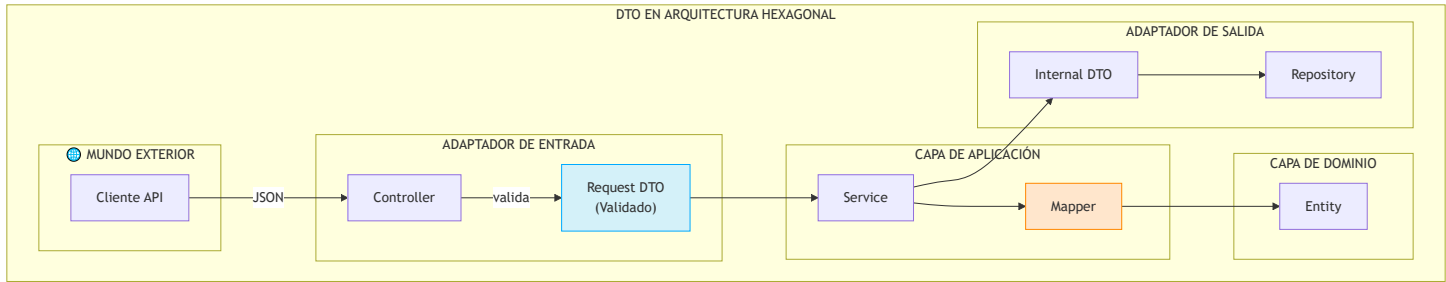
```

```

export class VehiculoMapper {
  static toResponse(vehiculo: Vehiculo): VehiculoResponse {
    return {
      id: vehiculo.getId(),
      marca: vehiculo.getMarca(),
      modelo: vehiculo.getModelo(),
      anio: vehiculo.getAnio(),
      precioBase: vehiculo.getPrecioBase().cantidad,
      precioFinal: vehiculo.calcularPrecioFinal().cantidad,
      tipo: vehiculo.getTipo(),
      stock: vehiculo.getStock(),
      estado: vehiculo.getStock() > 0 ? 'DISPONIBLE' : 'AGOTADO',
      especificaciones: vehiculo.getEspecificaciones()
    };
  }
}

```

7. DTO en Arquitectura Hexagonal



Ubicación de DTOs en Hexagonal

Capa	DTO	Responsabilidad
Infraestructura (Entrada)	Request DTO	Validar datos de entrada
Aplicación	Mapper	Transformar entre DTO y Entidad
Aplicación	Response DTO	Formatear datos de salida
Infraestructura (Salida)	Internal DTO	Comunicarse con repositorios

8. Validación de DTOs

Validación con Clase Validadora

```
// 📁 application/dtos/validators/employee.validator.ts

/**
 * EmployeeValidator
 * =====
 * Centraliza las validaciones de DTOs de empleados.
 * Separa la validación de la transformación.
 */
export class EmployeeValidator {

    private static readonly MIN_CEDULA_LENGTH = 10;
    private static readonly MAX_SALARY = 1_000_000;
    private static readonly MIN_SALARY = 0;

    /**
     * Valida un CreateEmployeeRequest
     * @throws {ValidationError} Si el request es inválido
     */
    static validateCreateRequest(request: CreateEmployeeRequest): ValidationResult {
        const errors: string[] = [];

        // Validar cédula
        if (!request.cedula) {
            errors.push('La cédula es requerida');
        } else if (request.cedula.trim().length < this.MIN_CEDULA_LENGTH) {
            errors.push(`La cédula debe tener al menos ${this.MIN_CEDULA_LENGTH} caracteres`);
        } else if (!/^[0-9]+$/.test(request.cedula)) {
            errors.push('La cédula solo debe contener números');
        }

        // Validar nombres
        if (!request.nombres) {
            errors.push('Los nombres son requeridos');
        } else if (request.nombres.trim().length < 2) {
            errors.push('Los nombres deben tener al menos 2 caracteres');
        }

        // Validar salario
```



```

    if (request.salario === undefined || request.salario === null) {
        errors.push('El salario es requerido');
    } else if (request.salario < this.MIN_SALARY) {
        errors.push(`El salario no puede ser negativo (mínimo: ${this.MIN_SALARY})`);
    } else if (request.salario > this.MAX_SALARY) {
        errors.push(`El salario no puede exceder ${this.MAX_SALARY.toLocaleString()}`);
    }

    return {
        isValid: errors.length === 0,
        errors
    };
}

/**
 * Valida un UpdateEmployeeRequest
 */
static validateUpdateRequest(request: UpdateEmployeeRequest): ValidationResult {
    const errors: string[] = [];

    // Validar que haya al menos un campo
    if (!request.nombres && request.salario === undefined && !request.departamento) {
        errors.push('Debe proporcionar al menos un campo para actualizar');
    }

    // Validar nombres si está presente
    if (request.nombres && request.nombres.trim().length < 2) {
        errors.push('Los nombres deben tener al menos 2 caracteres');
    }

    // Validar salario si está presente
    if (request.salario !== undefined) {
        if (request.salario < this.MIN_SALARY) {
            errors.push(`El salario no puede ser negativo (mínimo: ${this.MIN_SALARY})`);
        } else if (request.salario > this.MAX_SALARY) {
            errors.push(`El salario no puede exceder ${this.MAX_SALARY.toLocaleString()}`);
        }
    }

    return {
        isValid: errors.length === 0,
        errors
    };
};

```

```
    }  
}  
  
// =====  
// USO EN EL SERVICE  
// =====  
  
export class EmployeeService {  
    async createEmployee(request: CreateEmployeeRequest): Promise<Employee> {  
        //  Validación centralizada  
        const validation = EmployeeValidator.validateCreateRequest(request);  
        if (!validation.isValid) {  
            throw new ValidationError(validation.errors);  
        }  
  
        // Resto del código...  
    }  
}
```

9. Transformación: Mapeo de DTOs

Mapeo Manual vs Automático

```
// =====  
// MANUAL (Recomendado para control total)  
// =====  
  
class ManualMapper {  
    static toResponse(employee: Employee): EmployeeResponse {  
        return {  
            id: employee.getId(),  
            cedula: employee.getCedula(),  
            nombres: employee.getNombres(),  
            salario: employee.getSalario(),  
            departamento: employee.getDepartamento(),  
            createdAt: employee.getCreatedAt().toISOString(),  
            updatedAt: employee.getUpdatedAt().toISOString(),  
            activo: employee.isActive()  
        };  
    }  
}  
  
// =====  
// AUTOMÁTICO (Con librería class-transformer)  
// =====  
  
import { plainToClass, classToPlain } from 'class-transformer';  
  
class AutoMapper {  
    static toResponse(employee: Employee): EmployeeResponse {  
        return plainToClass(EmployeeResponse, employee, {  
            excludeExtraneousValues: true, // Solo campos con @Expose()  
            strategy: 'excludeAll'  
        });  
    }  
}
```

Mapecto con Validación

```
// 📁 application/dtos/mappers/employee.mapper.ts
```

```
import { Employee } from '../../../domain/entities/employee.entity.js';
import { EmployeeValidator } from '../validators/employee.validator.js';

export class EmployeeMapper {

    /**
     * Transforma y valida: CreateEmployeeRequest → Entidad
     * @throws {ValidationError} Si los datos son inválidos
     */
    static toEntity(request: CreateEmployeeRequest): Omit<EmployeeData, 'id' | 'createdAt' | 'updatedAt'> {
        // 1. Validar
        const validation = EmployeeValidator.validateCreateRequest(request);
        if (!validation.isValid) {
            throw new ValidationError(validation.errors);
        }

        // 2. Transformar
        return {
            cedula: request.cedula.trim(),
            nombres: request.nombres.trim(),
            salario: request.salario,
            departamento: request.departamento || 'Sin asignar'
        };
    }

    /**
     * Entidad → Response DTO (con opciones de formato)
     */
    static toResponse(employee: Employee, options?: ResponseOptions): EmployeeResponse {
        const response: EmployeeResponse = {
            id: employee.getId(),
            cedula: options?.maskCedula ? this.maskCedula(employee.getCedula()) : employee.getCedula(),
            nombres: employee.getNombres(),
            salario: employee.getSalario(),
            departamento: employee.getDepartamento(),
            createdAt: employee.getCreatedAt().toISOString(),
            updatedAt: employee.getUpdatedAt().toISOString(),
            activo: employee.isActive()
        };
    }
}
```

```
        return response;
    }

    /**
     * Oculta la cédula parcialmente
     */
    private static maskCedula(cedula: string): string {
        if (cedula.length <= 4) return cedula;
        return `***-${cedula.slice(-4)}`;
    }
}
```

10. Ventajas y Desventajas

Ventajas



Desventajas

Desventaja	Impacto	Mitigación
Código adicional	Más archivos que mantener	Usar mappers automáticos
Sobrecarga de transformación	Conversión de datos	Optimizar mapeos
Duplicación de estructura	Entidad y DTO similares	Mantener sincronizados

Desventaja	Impacto	Mitigación
Curva de aprendizaje	Nuevo concepto	Documentar bien

11. Anti-patrones: Qué NO Hacer

ANTI-PATRONES A EVITAR

✗ DTO con LÓGICA DE NEGOCIO

El DTO solo debe tener datos, no métodos de negocio

✗ DTO GIGANTE

Más de 20 campos en un DTO

✗ DTO con VALIDACIONES DE DOMINIO

Solo validaciones de formato, no reglas de negocio

✗ REUTILIZAR DTOs para diferentes casos

Crear DTOs específicos para cada caso

✗ MAPEO MANUAL en cada controlador

Centralizar el mapeo en Mappers

Ejemplo de Anti-patrón

// ❌ ANTI-PATRÓN: DTO con lógica de negocio

```
class BadEmployeeDTO {
  id: string;
  nombres: string;
  salario: number;

  // ❌ Esto debería estar en la ENTIDAD, no en el DTO
  calcularBonificacion(): number {
    return this.salario * 0.10; // Regla de negocio en DTO
  }

  // ❌ Validación de dominio en DTO
  validarEdad(edad: number): boolean {
    if (edad < 18) return false; // Regla de dominio
    return true;
  }
}
```

// ❌ ANTI-PATRÓN: DTO que duplica entidad y se usa para todo

```
interface UniversalDTO {
  // Usado para crear, actualizar, listar, todo...
  id?: string;
  cedula?: string;
  nombres?: string;
  salario?: number;
  createdAt?: string;
  updatedAt?: string;
  // ... y muchos campos opcionales
}
```

// ❌ ANTI-PATRÓN: Mapeo manual en cada controlador

```
class BadController {
  createEmployee(req, res) {
    // ❌ Mapeo manual y repetido
    const request = {
      cedula: req.body.cedula,
      nombres: req.body.nombres,
      salario: req.body.salario
    }
  }
}
```

```
};
```

```
// ...
```

```
}
```

```
}
```

Versión Correcta

//  DTO limpio: SOLO DATOS

```
interface CreateEmployeeRequest {  
    cedula: string;  
    nombres: string;  
    salario: number;  
}
```

//  DTOs específicos para cada caso

```
interface EmployeeResponse {  
    id: string;  
    cedula: string;  
    nombres: string;  
    salario: number;  
}
```

```
interface EmployeeListResponse {  
    data: EmployeeResponse[];  
    meta: PaginationMeta;  
}
```

//  Mapeo centralizado en el Service

```
class EmployeeService {  
    async createEmployee(request: CreateEmployeeRequest): Promise<EmployeeResponse> {  
        // Validación  
        const validation = this.validator.validate(request);  
        if (!validation.isValid) {  
            throw new ValidationError(validation.errors);  
        }  
  
        // Crear entidad  
        const employee = Employee.create(  
            request.cedula,  
            request.nombres,  
            request.salario  
        );  
  
        // Persistir  
        await this.repository.save(employee);  
    }  
}
```

```
        // Mapear a respuesta
        return this.mapper.toResponse(employee);
    }
}
```

12. Ejercicios Prácticos

Ejercicio 1: Crear DTOs para el Sistema de Concesionaria

```
// TODO: Crea los DTOs para:
// 1. CreateVehiculoRequest (marca, modelo, precio, stock)
// 2. VehiculoResponse (id, marca, modelo, precio, estado)
// 3. CreateCotizacionRequest (vehiculoId, plazo, entrada, cliente)
// 4. CotizacionResponse (id, vehiculo, cuotaMensual, totalInteres)
```

Ejercicio 2: Implementar el Mapper

```
// TODO: Completa el mapper

class VehiculoMapper {
    static toResponse(vehiculo: Vehiculo): VehiculoResponse {
        // Implementar la transformación
    }

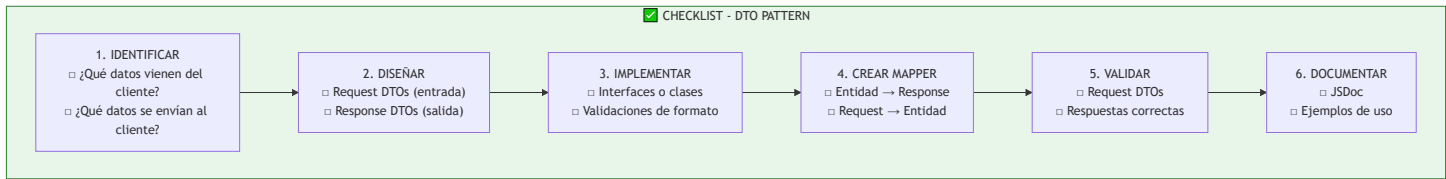
    static toEntity(request: CreateVehiculoRequest): Partial<VehiculoData> {
        // Implementar la transformación inversa
    }
}
```

Ejercicio 3: Validación de DTOs

```
// TODO: Implementa la validación de CreateCotizacionRequest
// Reglas:
// - vehiculoId: requerido, no vacío
// - plazo: entre 12 y 72 meses
// - entrada: 10% - 50% del precio del vehículo
// - cliente: nombre, email válido, teléfono

class CotizacionValidator {
    static validate(request: CreateCotizacionRequest): ValidationResult {
        // Tu implementación aquí
    }
}
```

13. Checklist de Implementación



Preguntas Clave para Decidir

Pregunta	Respuesta	Acción
¿Estás exponiendo datos internos?	Sí	Usar DTO
¿Necesitas diferentes vistas de datos?	Sí	Usar DTOs específicos
¿Tu API cambia cuando cambia la entidad?	Sí	Usar DTO
¿Quieres ocultar datos sensibles?	Sí	Usar DTO



Resumen Final

